Web Scraping

Fundamentos y Herramientas

Aplicaciones de Ciencia de Datos

- ## Clase 02: Web Scraping
- ## Universidad Nacional del Oeste

  Especialización en Ciencia de Datos

Agenda de la Clase

¿Qué es Web Scraping?

Estructura HTML: Las Bases

Selectores: XPath vs CSS

Librerías en R para Scraping

Librerías en Python para Scraping

Herramientas Avanzadas: Selenium

Servidores MCP: Puppeteer y Playwright

Consideraciones Éticas y Legales

Casos Prácticos Regionales

¿Qué es Web Scraping?

Definición

Técnica de extraer datos de sitios web de forma automatizada, simulando la navegación humana a gran escala.

¿Por qué es importante?

- 90% de los datos mundiales se generaron en los últimos 2 años
- Fuentes públicas: Gobiernos, instituciones, medios
- Datos en tiempo real: Precios, stocks, noticias

Casos de Uso

- Monitoreo de precios en e-commerce
- Análisis de sentimientos en redes sociales
- Recolección de datos gubernamentales
- Investigación académica y periodística

Estructura HTML: Entendiendo el DOM

Document Object Model (DOM)

El DOM es la representación estructurada de un documento HTML como un árbol de objetos.

Elementos Clave:

- Tags: , <p>, <h1>, etc.
- Atributos: class, id, href, etc.
- Jerarquía: Elementos padre, hijo, hermanos
- Contenido: Texto dentro de los elementos

Ejemplo de estructura HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mi Página</title>
  </head>
  <body>
    <div class="contenedor">
      <h1 id="titulo">Título Principal</h1>
      <p class="descripcion">Este es un párrafo</p>
      <ul>
        <li class="item">Elemento 1</li>
        <li class="item">Elemento 2</li>
      </ul>
    </div>
  </body>
</html>
```

Inspeccionar HTML en el Navegador

Herramientas de Desarrollador

- Chrome/Edge: F12 o Ctrl+Shift+I
- Firefox: F12 o Ctrl+Shift+I
- Safari: Cmd+Option+I

Funciones Útiles

- Inspeccionar elemento: Click derecho → "Inspeccionar"
- Consola: Para ejecutar JavaScript
- Network: Ver solicitudes HTTP
- Elements: Navegar el DOM

Práctica Inmediata

Vamos a inspeccionar: https://quotes.toscrape.com/

Selectores: XPath vs CSS

Selectores CSS: Simple y Efectivo

Sintaxis Básica:

```
/* Por tag */
p

/* Por clase */
.mi-clase

/* Por ID */
#mi-id

/* Combinaciones */
div.contenedor p.descripcion

/* Atributos */
a[href*="github"]

/* Pseudo-selectores */
li:first-child
tr:nth-child(2n)
```

Ejemplos Prácticos:

```
/* Todos los párrafos */
p

/* Primer elemento de lista */
li:first-child

/* Enlaces que terminan en ".pdf" */
a[href$=".pdf"]

/* Múltiples clases */
.precio, .costo, .valor
```

XPath: Más Potente, Más Complejo

¿Qué es XPath?

```
XML Path Language: Lenguaje para navegar elementos y atributos en XML/HTML.
```

Sintaxis Básica:

```
// Ruta absoluta
/html/body/div/p

// Ruta relativa
//div[@class="contenedor"]//p

// Por texto
//a[text()="Siguiente"]

// Contiene texto
//span[contains(text(), "precio")]

// Por posición
//li[1]         // Primer elemento
//li[last()]    // Último elemento
```

Operadores Avanzados:

```
// Operadores lógicos
//div[@class="item" and @id="especial"]
//p[contains(@class, "destacado") or @class="importante"]

// Navegación de ejes
//h1/following-sibling::p  // Hermanos siguientes
//td/parent::tr            // Elemento padre
//div/descendant::a        // Todos descendientes
```

CSS vs XPath: ¿Cuál elegir?

CSS Selectores

- ✓ Sintaxis simple y clara
- ✓ Más rápido en navegadores
- ✓ Familiar para desarrolladores web
- ✗ Limitado para navegación compleja

XPath

- ✓ Muy potente y flexible
- ✓ Navegación bidireccional
- ✓ Búsquedas complejas por texto
- ✗ Curva de aprendizaje más alta

Recomendación: CSS para casos simples, XPath para lógica compleja

Librerías y Herramientas

Librerías en R para Web Scraping

1. rvest (Recomendada para principiantes)

```
library(rvest)
library(dplyr)

pagina <- read_html("https://quotes.toscrape.com/")

quotes <- pagina %>%
  html_nodes(".quote") %>%
  html_text()
```

2. xml2 (Base de rvest)

```
library(xml2)

doc <- read_html("https://ejemplo.com")
nodes <- xml_find_all(doc, "//div[@class='contenido']")
texto <- xml_text(nodes)
```

3. httr2 (APIs y requests avanzados)

```
library(httr2)

response <- request("https://api.ejemplo.com/datos") %>%
  req_headers("User-Agent" = "Mi-Scraper/1.0") %>%
  req_perform()

datos <- response %>% resp_body_json()
```

Ejemplo Completo en R: Noticias

```
library(rvest)
library(dplyr)
library(stringr)

# URL de ejemplo (sitio de noticias argentino)
url <- "https://www.lanacion.com.ar/"

# Leer la página
pagina <- read_html(url)

# Extraer titulares (FUNCIONA)
titulares <- pagina %>%
  html_nodes(".title, .subhead") %>%
  html_text() %>%
  str_trim()

# Ver cuántos conseguimos
length(titulares)

# Versión corregida - sin na.rm
enlaces <- pagina %>%
  html_nodes("a") %>%
  html_attr("href") %>%
  .[!is.na(.) & str_detect(., "/nota/|/politica/|/economia/")] %>%
  head(length(titulares))

head(titulares)
```

Librerías en Python para Web Scraping

1. requests + BeautifulSoup

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

response = requests.get("https://quotes.toscrape.com/")
soup = BeautifulSoup(response.content, 'html.parser')

quotes = []
for quote in soup.find_all('div', class_='quote'):
    text = quote.find('span', class_='text').text
    author = quote.find('small', class_='author').text
    quotes.append({'texto': text, 'autor': author})

df = pd.DataFrame(quotes)
```

2. scrapy (Framework profesional)

```
import scrapy

class QuotesSpider(scrapy.Spider):
    name = 'quotes'
    start_urls = ['https://quotes.toscrape.com/']

    def parse(self, response):
        for quote in response.css('div.quote'):
            yield {
                'text': quote.css('span.text::text').get(),
                'author': quote.css('small.author::text').get(),
            }
```

3. httpx (Asíncrono moderno)

```
import httpx
import asyncio

async def fetch_data():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.ejemplo.com")
        return response.json()
```

Herramientas Avanzadas: Selenium

¿Cuándo usar Selenium?

```
✓ JavaScript rendering: Contenido dinámico
✓ Interacciones: Clicks, formularios, scroll
✓ Páginas SPA (Single Page Applications)
✓ Autenticación: Login requerido
```

Python + Selenium:

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

options = webdriver.ChromeOptions()
options.add_argument('--headless')
driver = webdriver.Chrome(options=options)

try:
    driver.get("https://ejemplo.com")
    element = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.CLASS_NAME, "contenido"))
    )
    datos = driver.find_elements(By.CSS_SELECTOR, ".item")
finally:
    driver.quit()
```

R + RSelenium (CORREGIDO):

```r
library(RSelenium)

driver <- rsDriver(browser = "chrome", chromever = NULL)
remote_driver <- driver[["client"]]

remote_driver$navigate("https://ejemplo.com")
elements <- remote_driver$findElements(using = "css", ".item")

remote_driver$close()
driver[["server"]]$stop()
```

Servidores MCP: Automatización Inteligente

¿Qué son los MCPs?

```
Model Context Protocol: Conectan herramientas de automatización web con IA.
```

Puppeteer (JavaScript):

```
const puppeteer = require('puppeteer');

class PuppeteerMCP {
    async navigate(url) {
        const browser = await puppeteer.launch();
        const page = await browser.newPage();
        await page.goto(url);
        const content = await page.content();
        await browser.close();
        return content;
    }
}
```

Playwright (Python - Multi-navegador):

```
from playwright.sync_api import sync_playwright

class PlaywrightMCP:
    def __init__(self):
        self.playwright = sync_playwright().start()
        self.browser = self.playwright.chromium.launch()

    def navigate_and_extract(self, url, selector):
        page = self.browser.new_page()
        page.goto(url)
        page.wait_for_selector(selector)
        elements = page.query_selector_all(selector)
        return [el.text_content() for el in elements]
```

Ventajas de Puppeteer y Playwright

Puppeteer

- Control total del navegador Chrome
- Interceptar requests y responses
- Generar PDFs de páginas web
- Automatización completa de UI

Playwright

- Multi-navegador: Chrome, Firefox, Safari
- Auto-waiting: Espera inteligente de elementos
- Testing: Diseñado para pruebas automatizadas
- Network handling: Manejo avanzado de red

Ambos permiten integración con IA para scraping inteligente

Ética y Legalidad

Consideraciones Éticas y Legales

Aspectos Legales

- robots.txt: Siempre revisar /robots.txt
- Términos de Servicio: Leer ToS del sitio
- Rate Limiting: No sobrecargar servidores
- Copyright: Respetar propiedad intelectual

Aspectos Éticos

- Propósito legítimo: Investigación, periodismo
- Datos públicos: Solo info accesible públicamente
- Privacidad: No datos personales sin consentimiento
- Transparencia: Identificarse como bot

⚠️ **El scraping responsable protege tanto al usuario como al sitio web**

Buenas Prácticas en Código

Scraping Responsable:

```
import time
import random
import requests

def scrape_responsibly(urls):
    for url in urls:
        # Delay aleatorio entre requests (1-3 segundos)
        time.sleep(random.uniform(1, 3))

        # Headers que simulan navegador real
        headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'
        }

        try:
            response = requests.get(url, headers=headers, timeout=10)
            # Procesar datos...
        except requests.exceptions.RequestException as e:
            print(f"Error en {url}: {e}")
            continue
```

Casos Prácticos Regionales

Caso 1: Monitoreo de Precios - MercadoLibre

```
library(rvest)
library(dplyr)

monitorear_precios <- function(busqueda) {
  url <- paste0("https://listado.mercadolibre.com.ar/", busqueda)
  pagina <- read_html(url)

  productos <- pagina %>%
    html_nodes(".ui-search-result__content") %>%
    map_dfr(~ {
      titulo <- .x %>% html_node(".ui-search-item__title") %>%
                html_text()
      precio <- .x %>% html_node(".price-tag-amount") %>%
                html_text()
      enlace <- .x %>% html_node("a") %>% html_attr("href")

      data.frame(titulo, precio, enlace)
    })

  return(productos)
}

# Uso
notebooks <- monitorear_precios("notebook-i5")

# NOTA: Selectores pueden cambiar. Verificar periódicamente.
```

Caso 2: Datos Gubernamentales - INDEC

```
import requests
import pandas as pd

def obtener_datos_indec():
    # API del INDEC para índices económicos
    url = "https://apis.datos.gob.ar/series/api/series"

    params = {
        'ids': '148.3_VARIACION_MENSUAL_2016',  # IPC
        'limit': 100,
        'format': 'json'
    }

    response = requests.get(url, params=params)
    data = response.json()

    df = pd.DataFrame(data['data'])
    return df

# Análisis de inflación
inflacion = obtener_datos_indec()
print(inflacion.head())
```

Proyecto: Dashboard de Datos Regionales

Objetivo

Sistema de monitoreo automático combinando:

- Precios de productos básicos (MercadoLibre)
- Noticias económicas locales
- Datos gubernamentales (INDEC, BCRA)
- Agricultura y clima regional

Arquitectura

Scheduler → Scrapers → Database → Dashboard

Componentes Clave

1. Scraper de precios y noticias
2. Integración con APIs gubernamentales
3. Base de datos PostgreSQL
4. Dashboard interactivo (Shiny/Dash)

Herramientas de Desarrollo

IDEs Recomendados:

```
✓ RStudio: Excelente para R + Shiny
✓ VS Code: Universal con extensiones
✓ PyCharm: Especializado en Python
✓ Jupyter: Notebooks para experimentación
```

Debugging en R:

```r
library(rvest)
pagina <- read_html("https://ejemplo.com")

# Ver estructura HTML
pagina %>% html_structure()

# Probar selectores
pagina %>% html_nodes("div.clase") %>% length()
```

Debugging en Python:

```python
from bs4 import BeautifulSoup
import requests

response = requests.get("https://ejemplo.com")
soup = BeautifulSoup(response.content, 'html.parser')

print(soup.prettify())
print(len(soup.find_all('div', class_='item')))
```

Manejo de Errores:

```python
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def crear_sesion_robusta():
    session = requests.Session()
    retry_strategy = Retry(
        total=3,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["HEAD", "GET", "OPTIONS"]  # ✓ CORREGIDO
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("http://", adapter)
    session.mount("https://", adapter)
    return session
```

Recursos y Referencias

Documentación Oficial

rvest: https://rvest.tidyverse.org/

BeautifulSoup: crummy.com/software/BeautifulSoup/

Selenium: selenium-python.readthedocs.io/

Sitios para Práctica

Quotes to Scrape: quotes.toscrape.com/

Books to Scrape: books.toscrape.com/

Scrape This Site: scrapethissite.com/

Datasets Argentinos

Datos Argentina: datos.gob.ar/

INDEC: indec.gob.ar/

BCRA: bcra.gob.ar/

¡Gracias por su atención!

"Some tasks are just too tedious—or too important!—to leave to humans."
— Michael Schrenk, *Webbots, Spiders, and Screen Scrapers*